

System design workbook

Table of contents

<i>System Design.....</i>	<i>4</i>
<i>General.....</i>	<i>4</i>
Introduction	4
Learning goals	4
Topics	5
Grading.....	6
Entry Requirements	6
Grading.....	7
Assessment.....	7
Grading.....	8
Summary.....	8
Generatieve AI.....	9
Reading guide	10
Tips.....	10
<i>Week 1 (C4 layer 1 context, layer 2 container).....</i>	<i>11</i>
Theory.....	11
Workshop.....	11
Exam assignment.....	12
<i>Week 2 (c4 layer 3 component).....</i>	<i>13</i>
Theory.....	13
Workshop.....	13
Exam assignment.....	13
<i>Week 3 (c4 laag 4 code)</i>	<i>14</i>
Theory.....	14
Workshop.....	14
Exam assignment.....	16
<i>Week 4 (Van ontwerp na code).....</i>	<i>17</i>

Theory.....	17
Workshop.....	17
Exam assignment.....	18
<i>Week 5 (Code analyse)</i>	<i>19</i>
Theory.....	19
Workshop.....	19
Exam assignment.....	19
<i>Week 6 (Refactoring: ontwerpen).....</i>	<i>20</i>
Theorie	20
Workshop.....	20
Exam assignment.....	21
<i>Week 7 (Refactoring: iterating).....</i>	<i>22</i>
Theory	22
Workshop.....	22
Exam assignment.....	22
<i>Exam assignment 1 – Battle Engine</i>	<i>23</i>
Context.....	23
Assignment	23
Requirements	25
Interface	25
Functional requirements	26
<i>Exam assignment 2 – 3Dprintscheduler.....</i>	<i>28</i>
<i>3D printer casus</i>	<i>30</i>
Introduction	30
Background information	30
Basic program operation	32
Requirements	33
Business requirements.....	33
User requirements	33
System Requirements	33

Testing	34
Diagrams	34
<i>Reflection Report</i>	<i>35</i>
Partial Reflection	35
eJournal Environment	36
How to submit your work in the eJournal environment	36
<i>Bibliography</i>	<i>39</i>

System Design

General

Introduction

This course focuses on software design. We use the C4 model, which involves working across four levels of abstraction: from a global overview down to detailed design. You'll learn how to apply the appropriate UML diagrams at each level of abstraction, enabling you to communicate effectively and convincingly about software design in a professional context.

In addition to designing new software, we also focus on redesigning and improving (refactoring) existing software. You'll analyse existing code to identify the strengths and weaknesses of the current design. Based on this analysis, you'll learn how to create an improved redesign. In this way, you'll develop the ability to make well-reasoned and substantiated decisions when designing new solutions or redesigning existing ones.

The course is taught over 7 weeks. Each week consists of three classes, structured as follows:

Per week:	What you do:	Why?:
Lesson 1 – Lecture	→ Listen, process, understand	→ Gain knowledge
Lesson 2 – Workshop *!	→ Small exercises	→ Apply knowledge to gain competence
Lesson 3 – Lab/Practical	→ Exam assignment (assignment 1 w1-4, assignment 2 w5-8)	→ Use competence to solve complex real world scenario's

***! entry requirement:**

- You must check off all workshop assignments
- Submitted & complete: exam assignments



7

FIGURE 1 EACH WEEK CONSISTS OF THREE CLASSES: A THEORY SESSION, A WORKSHOP, AND A CLASS DEDICATED TO WORKING ON THE EXAM ASSIGNMENTS.

Learning goals

Below are the learning objectives, which are aligned with the HBO–I model (SW = Software, DES = Design, REA = Realization, ANA = Analysis) (HBO–i, 2025; HBO–i, 2025).

(SW/DES/2) You create software designs that correspond to the different abstraction levels of the C4 model, making use of UML diagrams.

(SW/DES/2) You justify your design choices in a professional context.

(SW/REA/2) You develop new code and refactor existing code into modular, maintainable software based on programming principles such as SOLID.

(SW/ANA/2) You substantiate shortcomings in code quality and design principles by analyzing existing code.

Professional skills

Professional skill (PS)	description
OV > Methodical Problem-Solving Approach (MPA)	MPA: You identify relevant issues and/or opportunities, determine which knowledge is lacking, and plan research follow-up steps in a structured and critical way, selecting methods appropriate to the problem at hand.
OV > Research (OZK)	OZK: You conduct research with an open mindset in a substantiated, pragmatic, structured, and critical manner.
OV > Solution (OPL)	OPL: You apply the information obtained from research within the context of the problem and make proposals based on this information, while remaining critical and open to alternative ideas and approaches.
PL > Personal Development (POW)	You substantiate your study and career choices and adjust your own learning development with reasoned reflection (for example, through reflection and/or feedback).
DI > Communication (COM)	You adapt your communication thoughtfully and purposefully to your target audience(s).
DI > Collaboration (SAM)	You collaborate consciously, constructively, and in an appropriate form, taking responsibility for your own contribution (for example, in interdisciplinary and/or intercultural contexts) and for the final result.

Topics

This course covers the following topics:

- Designing software at different levels of abstraction using the **C4 model**.
- Using professional **diagrams**: including deployment, component, class, and use-case diagrams.
- **Justifying** and substantiating design decisions.
- **Analyzing** and evaluating existing software.
- Redesigning and **refactoring** existing software.
- **Using (your own) designs** to develop new software.

Grading

This course is completed in a **pair** of 2. You will submit joint work together, supplemented by individual assignments. Both team members are expected to be familiar with all jointly **submitted work**.

Assessment consists of a **pair assessment** and an **individual reflection report**. In addition, there are several **entry requirements** that must be met before you are allowed to participate in the assessment (and therefore receive a grade).

Entry Requirements

All the following entry requirements must be met:

1. Practical assignments (individual)

- Each week, assignments will be provided that can be completed during the practical sessions (and as homework). All assignments must be signed off by the lecturer. This can be done during the practical classes (in lesson 2, only the material from that week can be signed off; in lesson 3, material from any week may be signed off).

Note: The practical assignments are not graded, and your solutions do not need to be perfect. However, they must demonstrate a genuine effort.

2. Exam assignments (pairs)

- In addition to the practical assignments, you will work on two exam assignments. For each assignment, you will produce a Technical Design document (TD) and refactor existing code. Both Technical Design documents and the refactored code must be complete and submitted before the assessment begins. During the assessment, these documents and the associated code can be consulted.

The exam assignments are structured as follows:

- **Exam Assignment 1 (TD):**
 - **Technical Design Document 1 (TD 1)** (completed during weeks 1–4). This assignment focuses on designing new software (Case 1: Battle Engine).
- **Exam Assignment 2 (TD & Code):**
 - **Technical Design Document 2 (TD 2)** (completed during weeks 5–7). This assignment focuses on analyzing and redesigning existing code (Case 2: 3D Printer).
 - **Refactored code** based on the design described in TD 2.

Grading

Once the entry requirements have been met, the course is assessed through an individual reflection report (consisting of three partial reflection submissions) and a joint assessment (completed in pairs).

1. Reflection report (individual)

Each student writes an individual reflection report by submitting a partial reflection every two weeks, resulting in three reflection submissions. It is important that these reflections clearly demonstrate your individual learning process. Therefore, you are strongly encouraged to submit each reflection by the recommended submission date.

These reflections must be submitted through the eJournal environment.

See the section Reflection Report and eJournal Environment in this document for more information. Refer to the rubric on Brightspace (or in the eJournal environment) to see how the reflections are assessed.

2. Assessment (pair)

The assessment consists of a presentation followed by a short defense, as described below.

Assessment

The assessment covers Exam Assignments 1 and 2. It takes place at the end of the quartile in which the course is offered (see the schedule on Brightspace). Your group (pair) will receive an invitation in advance containing the date, time, and location of the assessment. The assessment lasts approximately 20–30 minutes and is completed together.

Before the assessment, you will prepare a presentation of 5–8 minutes. After the presentation, the lecturers will ask questions about your designs, design decisions, and the rationale behind them (the defense). You will also be asked to demonstrate parts of the refactored code (Exam Assignment 2, **3DPrinterSchedule**), so make sure you have the code available.

Please note the following:

- A presentation template is available on Brightspace. Your presentation should cover the key aspects of the exam assignments. **The presentation is not a substitute for submitting the exam assignments themselves.**
- Aim for a presentation of **5–8 minutes**. Please note that presentations will be stopped after **10 minutes**.
- Both team members must be familiar with all submitted work and both must play an active role during the presentation and defense.
- Make sure you have the code for **Exam Assignment 2 (3DPrinterSchedule)** available. You should also have **TD 1**, **TD 2**, and your individual reflection reports available (for example, on a laptop).

- You can expect questions such as: *"How did you arrive at this design?"*, *"Can you explain design choice X?"*, *"Can you show the code that resulted from design Y?"*, and *"Which alternatives did you consider?"* We assess not only the result, but also the design process.
- In the unfortunate event that one team member meets the entry requirements and the other does not, the eligible student may complete the assessment individually.

Grading

Once all entry requirements have been met, a final grade will be determined based on the assessment in combination with the submitted reflection reports. Please refer to the rubric on Brightspace for the assessment criteria.

Summary

Below is a summary of what must be delivered to successfully complete the **System Design** course:

Deadline (end of the mentioned week)	Deliverable	Individual/Group (submission location)	Requirement/ Assessment Component	Explanation	Consists of
8	All workshop assignments signed off	Individual (during class)	Entry Requirement 1	Completion of the workshop assignments, including the associated code. Have the assignments signed off by your lecturer during the workshop sessions.	Assignment solutions & associated code
8	Exam assignments	Group (pairs) (Brightspace)	Entry Requirement 2	Create a TD for Case 1 (Combat Engine) and Case 2 (3D Printer). For Case 2, the existing code must also be refactored. Everything must be submitted and complete.	TD 1 (Case 1), TD 2 (Case 2), Refactored code (Case 2). Note that both TDs must include a Design Justification chapter.
Partial-Reflection 1: 2 Partial-Reflection 2: 4 Partial-Reflection 3: 6	Iterative reflection report	Individual (eJournal)	Assessment	You write a reflection report iteratively. It is recommended to submit one reflection every two weeks.	3 partial reflection submissions
To be determined in week 9 (see Brightspace schedule)	Assessment	Group (pairs) (on location)	Assessment	Give a presentation of a maximum of 8 minutes, followed by answering questions from the lecturers (10-20 minutes).	Presentation and defense. Additionally, the TDs and refactored code from Cases 1 and 2 may be included in the assessment.

Deliverables

Ultimately, the above results in the following deliverables:

- Individual**
 - Completed and signed off workshop assignments (including code) (Entry Requirement 1)
 - Reflection report (consisting of 3 partial-reflection submissions)
- Joint**

- Exam assignments (Entry Requirement 2)
 - TD 1
 - TD 2
 - Refactored code (associated with TD 2)
- Assessment: Presentation + defense

Note: By following the workbook systematically each week, the above deliverables will be completed on time according to the schedule.

Generatieve AI

You may use Generative AI transparently and responsibly according to Saxion's guidelines for AI use by students. Your work must include a justification of the use of AI in your process according to the APA guidelines. You always remain responsible for the content of your work.

If there is any doubt about the authenticity of your work, your lecturer may schedule an additional interview with you or assign an additional task. The examination committee may also be involved.

It is important that the designs are created iteratively. Generative AI may be used as a source of inspiration, but the actual thinking and design process must come from yourself.

Generative AI used in the System Design course materials

Generative AI was used for the following parts during the development of the System Design course materials:

- As inspiration for some of the workshop assignments
- Spelling and grammar correction
- Support in translating from Dutch to English

In all cases, all generated output was checked and validated by a human (and adjusted where necessary). In no case was the vision or objective outsourced.

Retake

For the retake, the same criteria apply as for the regular submission, except for the **iterative reflection report** (see below).

A new assessment must also be completed. The original (individual) entry requirements remain applicable:

- **Workshop assignments:** The workshop assignments must have been signed off. Two additional moments will be scheduled where you can still have the workshop assignments signed off. An announcement will be posted on Brightspace regarding these moments.
- **Reflection report:** Instead of 3 partial-reflection submissions, you will write an extensive retake reflection report. See the Brightspace environment for the exact requirements (and assessment criteria) for this retake reflection report.

Reading guide

This workbook is the reference document used as a guideline throughout **the System Design** course. The learning material is divided across the 8 weeks of the course. Each week is divided into:

1. **Theory:** The material discussed during the lecture (lesson 1).
2. **Workshop assignments:** The assignments that must be signed off. During the workshop session (lesson 2) (and as homework), you will work on these assignments.
3. **Exam assignment:** The exam assignments are designed to align with the learning material of the corresponding week. By keeping up with the work systematically, you avoid a heavy workload at the end of the course. During the practical session (lesson 3) (and as homework), you will work on these assignments.
4. **Reflection report and eJournal:** The requirements for the reflection submissions, including an explanation of how they can be submitted through the eJournal environment.

Tips

Tools for creating UML diagrams:

- **PlantUML:** Can be used in IntelliJ / Markdown or online: <https://plantUML.com>
- **Draw.io:** An online tool for creating various types of diagrams: <https://app.diagrams.net>
- **On paper:** And take a photo later

Week 1 (C4 layer 1 context, layer 2 container)

Theory

Introduction/general

- Read the General section of the workbook to become familiar with the structure of this course.
- Form a team of two with a classmate and register as a pair on Brightspace.
- Schedule 3-5 hours per week to work together on this course.

C4 model

- Introduction to the C4 Model
 - Read: *Introduction, Abstractions/1. Software system, Abstractions/2. Container*. see: <https://c4model.com>

UML

- UML (Unified Modeling Language), see: <https://www.uml-diagrams.org>
 - Study: *Home page* (The Unified Modeling Language)
 - Study the following diagrams: use-case, deployment, component.
- Component diagram, read: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/> (until: *subsystems*)
- Deployment diagram, read: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/> (until: *Steps for Modeling an Embedded System*)

Workshop

Assignment 1 – C4 basics

Answer the following questions in your own words:

1. Briefly describe the four layers of the C4 model.
2. What is the advantage of designing from a global to a detailed level?
3. (optional) Which programming experiences have you had that might have gone better if you had created a design beforehand?

Assignment 2 – Applying C4

Below are several sentences; assign each one to the correct layer of the C4 model. The case describes a platform where users can buy, search for, and read books via a website or mobile app.

1. Order management handles shopping carts and payments.
2. A publisher provides new book data.
3. The class Book contains properties such as title, author, and price.
4. The database stores user data and book information.
5. A user searches for and buys books through the platform.
6. The search service processes queries and filters.
7. The class OrderService has methods *createOrder()* and *cancelOrder()*.
8. The platform connects with an external payment service for transactions.
9. The web app displays books and the shopping cart to the user.
10. The API server processes orders and manages communication with other systems.
11. The authentication component handles login and registration.
12. The class *UserRepository* retrieves user data from the database.

Assignment 3 – Use cases

Use the case below to create a use case diagram showing which actors exist and which actions they perform with the system. You may invent additional actions that seem logical. We are working at abstraction layer 1 (context) of the C4 model.

Case: The university wants to develop a new online library platform. Students and teachers should be able to easily find, reserve, and borrow both digital and physical books. Teachers should also be able to create and share reading lists with their students.

Assignment 4 – Deployment diagram

Use the case from Assignment 3 again. First, create a deployment diagram in which you design what hardware is needed to set up this system. Add notes explaining the considerations and choices you made.

Assignment 5 – Component diagram

Use the case from Assignment 3 again, along with the deployment diagram created in Assignment 4. Now, add the main components. You do not need to go into detail yet, for example, “Front-end” is sufficient; you do not need to divide it into subcomponents at this stage. Add interfaces where necessary. Include notes explaining the considerations and choices you made.

Exam assignment

Read exam assignment 1.

Using Exam Assignment 1, complete the following parts: objective, C4 layer 1, and C4 layer 2.

Week 2 (c4 layer 3 component)

Theory

C4: watch video: <https://youtu.be/x2-rSnhpw0g?si=Imt13UIWyPuc85Gp>

Component diagrams: watch video: <https://www.youtube.com/watch?v=Y2llwZT1VLE>

Alternative, read:

- Component diagram: <https://www.geeksforgeeks.org/software-engineering/component-based-diagram/>
- C4 read Abstractions/3. Component and Diagrams/1-4 (see: <https://c4model.com>)

Workshop

In the workshop, we will practice translating requirements into a design. We will work with the following case:

1. **User management**
 - A user can create an account and log in.
2. **Task management**
 - A user can add, edit, check off, and delete tasks.
 - Tasks have a title, an optional description, and a deadline.
 - Tasks can be categorized (e.g., work, study, personal).
3. **Notifications**
 - The system sends a reminder when a deadline is approaching.

Non-functional requirements

1. The app must be accessible via a web browser.
2. Data must be stored persistently.
3. The system must be user-friendly and easy to navigate.

Assignment 1 – C4 Context

Create a C4 Level 1 Context diagram that defines what the system is and identifies the other systems and users involved.

Assignment 2 – C4 Containers

Extend your C4 design with Level 2, focusing on your system. Define which containers your system consists of.

Assignment 3 – C4 Components

Extend your C4 design with Level 3, focusing on 2-3 containers of your system. Outline which components these containers contain.

Assignment 4 – From C4 to Component Diagram

Create a component diagram from the design you have made in assignment 3.

Exam assignment

Develop a design in C4 notation for levels 1-3. Then, add a component diagram for level 3 (see exam assignment 1).

Week 3 (c4 laag 4 code)

Theory

read:

- Class diagram: <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>
- Understand what level 4 in the C4 model represents and what level of detail it involves. Check the overview on the C4 website for this: <https://c4model.com> (*noot: The creators of the model intentionally left level 4 open for interpretation; in our case, we use UML class diagrams to represent it*).
- Make sure you are well familiar with these modeling techniques, as they form the foundation for the SOLID principles:
 - Encapsulation: <https://www.geeksforgeeks.org/java/encapsulation-in-java/>
 - Inheritance: <https://www.geeksforgeeks.org/java/inheritance-in-java/>
 - Abstractions: <https://www.geeksforgeeks.org/java/abstraction-in-java-2/>
 - Interfaces: <https://www.geeksforgeeks.org/java/interfaces-in-java/>
- Study the SOLID principles:
 - In this course, we focus on: Single Responsibility, Open/Closed, and Interface Segregation (Liskov and Dependency Injection are optional). See: <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>

Workshop

Assignment 1 – class diagrams

For this assignment, the Java project ‘gradingSystem’ (available on Brightspace) will be used.

We want to become familiar with the code. Therefore, create a class diagram following proper UML standards.

Do not generate it! (By drawing it yourself, you’ll need to go through the code manually, which helps you understand it better — this effect is lost if you generate it automatically.)

Assignment 2 – Working with software modeling techniques

For this assignment, we will use the Java project ‘**gradingSystem**’ (available on Brightspace).

We are going to extend the gradingSystem. The following requirements have been defined for this. **Before you start programming, first create a redesign!**

- The system must also support a **Coach**. A coach has:
 - A name
 - A method called: `public String helpPerson(...)`. This method can accept any person and returns the following string:
"Person [name_person] has been helped!"
- **To test:** create a few coaches and call the helpPerson method, passing another person as an argument.
- Add an **abstract method** to the Person class called: `public abstract String getDateOfBirth()`. What happens when you try to run the code now? Each type of person must return something different when this method is called:
 - **Teacher:** returns "Teacher, date of birth: " + [dateOfBirth]
 - **Coach:** returns "Coach, date of birth: " + [dateOfBirth]
 - **Student:** returns "Student, date of birth: " + [dateOfBirth]

- Some people must support the method `public int pay(...)`, but they should behave differently:
 - **Teacher:** returns [salary]
 - **Coach:** returns 3000
 - **Student:** does **not** support this method
- **To test:** call this method on a few coaches and teachers. How did you solve this problem?

Assignment 3 – Modeling a data model

In this assignment, you will model a **data model** for the Pokémon trading card game. This model must be able to store 0-* (zero to “infinite”) cards. Design a good solution for the following requirements. See an example of a Pokémon card below the requirements.

Requirements:

- The data model can store 0-* cards.
- The data model must support storing the following properties for a Pokémon card:
 - Name (example: *Charizard*)
 - Number of hitpoints (HP) (example: *120*)
 - A Pokémon type, represented as an elementType (example: *fire* (top right corner))
 - Abilities (the two text blocks on the card):
 - Each Pokémon has 1–2 abilities, which can be of type power or targetedAbility.
 - In the example, the first is a power (passive effect) called *Energy Burn*.
 - The second in the example is a targetedAbility called *Fire Spin*.
A targetedAbility always has 1–4 energy costs (represented as elementType) — in this case, 4× *fire*. It also has a name, a description, and a damage value indicating how much damage the ability deals.
 - A Pokémon can have a weakness and a resistance, both represented by an elementType.
 - For completeness: elementType is always one of the following:
fire, grass, water, psychic, fighting, electric, colorless.



FIGURE 1 POKEMON CARD

Exam assignment

Develop several class diagrams for C4 Level 4 (Code) (see also exam assignment 1) to design a solid data model. Use the correct UML notation for this. Discuss together what the best approach is to address certain problems from the requirements, and document the design decisions you made.

Week 4 (Van ontwerp na code)

Theory

- Study the SOLID principles:
 - This week we focus on: open/closed (Liskov en Dependency injection are optional). See: <https://www.geeksforgeeks.org/system-design/solid-principle-in-programming-understand-with-real-life-examples/>
- Study what design patterns are in general, see: <https://refactoring.guru/design-patterns>
- Study the facade pattern: <https://refactoring.guru/design-patterns/facade>
- Read about dependencies in java: <https://jenkov.com/tutorials/ood/understanding-dependencies.html>

Workshop

(optional) Assignment 1: Discuss your solutions from last week

Discuss with your partner the provided solutions (from last week's workshop assignments) shown during the lecture, and compare them to your own.

Note the main differences or similarities between your solution and the one discussed in class.

Assignment 2: Notification service

For this assignment, you will build a NotificationService. This service can send emails, SMS messages, and WhatsApp messages.

The current implementation looks as follows:

NotificationService

```

└── sendNotification(type: String, message: String): void

// if type == "email" → send email

// if type == "sms" → send SMS

// if type == "wa" → send whatsapp-message

```

Design a solution where the **NotificationService** can be configured once with a specific notification type, and can then send messages without the sender having to specify how they should be sent (for example, by calling `NotificationService.send("hello world")`). Design your solution in such a way that the **Open/Closed Principle** of SOLID is followed when deciding to add a new notification type (for example, `type == "tg" → send a Telegram message`).

Create a UML class diagram and sketch what the code (pseudo code) would approximately look like. You do not need to actually implement the code.

Assignment 3: facade class

For this assignment, use the 'gradingSystem' Java project again (use last week's version or download it again from Brightspace).

Question: How many dependencies does the Grading application currently have?

At the moment, the coupling between the application and the domain is too tight.

Task: Implement a Facade class between the application and the domain layer. You may refer to the lecture slides for tips on how to implement the facade. **For this assignment, you will actually write code.**

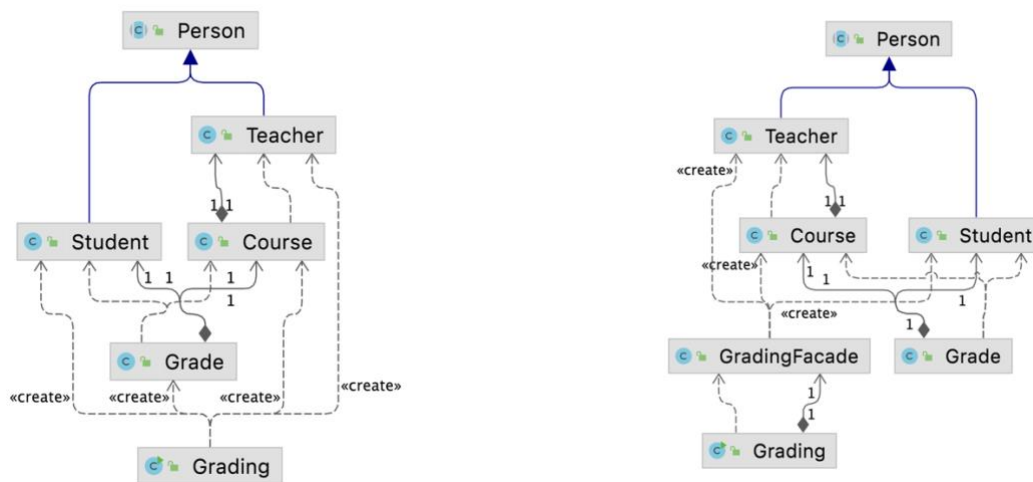


FIGURE 2– ON THE LEFT, THE SITUATION BEFORE THE FACADE: THE COUPLING BETWEEN THE APPLICATION AND THE DOMAIN IS VERY STRONG. ON THE RIGHT, THE DESIRED SITUATION, WHERE A FACADE SEPARATES THE DOMAIN FROM THE APPLICATION.

Exam assignment

Complete Exam Assignment 1. Add the SOLID principles to your design.

The goal is to integrate all requirements effectively into your design while also considering how adaptable and extensible your system is.

Week 5 (Code analyse)

Theory

Study code smells

- Study refactoring, read: 'What is refactoring' and all its subchapters, see: <https://refactoring.guru/refactoring/what-is-refactoring>
- Understand the different categories and get a feel for the different types of code smells. See <https://refactoring.guru/refactoring/smells>

Workshop

Assignment 1: Code smells

Analyze the code in the footballManager Java project individually. This project can be found on Brightspace. Add *//TODO* comments in the project at the locations where you find a code smell. After each *//TODO*, write the name of the code smell or briefly describe what is wrong with that part of the code. Tip: there are at least 20–25 code smells in the code!

For example: //TODO: this class has too many responsibilities.

Note: in the Java project, you will also find a file called README-EN.md, which explains the assignment again with a bit more information.

Assignment 2: Discuss code smells with your partner and reflect

Discuss with your teammate the code smells you found in Assignment 1. Describe the differences between your findings. Then briefly write down what you would do differently next time when working on a similar assignment.

Exam assignment

This week, you will analyze the code of the **3DScheduler**. For this, you will write the **first part of your technical design** for **Exam Assignment 2**, which includes the following:

- Describe the assignment using the C4 model, start with Levels 1 and 2. Describe what the system will be used for (and by whom). Then continue to Level 2. You may introduce new containers but make sure to justify them. Link the elements below to the appropriate layers of the C4 model.
- Analysis of the existing code – provide a clear overview of how the current code is structured.
- Code smells – identify and describe them.

Week 6 (Refactoring: design)

Theorie

Study the material on refactor-guru about: refactoring, the strategy pattern and the singleton pattern. Link:

<https://refactoring.guru/refactoring>

Make sure you understand which problems these patterns solve. It is also important to actually practice using these patterns (we will do this during the practical sessions and as part of the exam assignment).

Workshop

Assignment 1: Strategy pattern

Create a new Java project.

- Create a new Java project.
- Implement the data model as shown in the class diagram below (Figure 3).
- Make it possible for the user to add new collectors and drivers.
- Make it possible to change an employee's JobFunction.
- Make it possible to print all salaries.

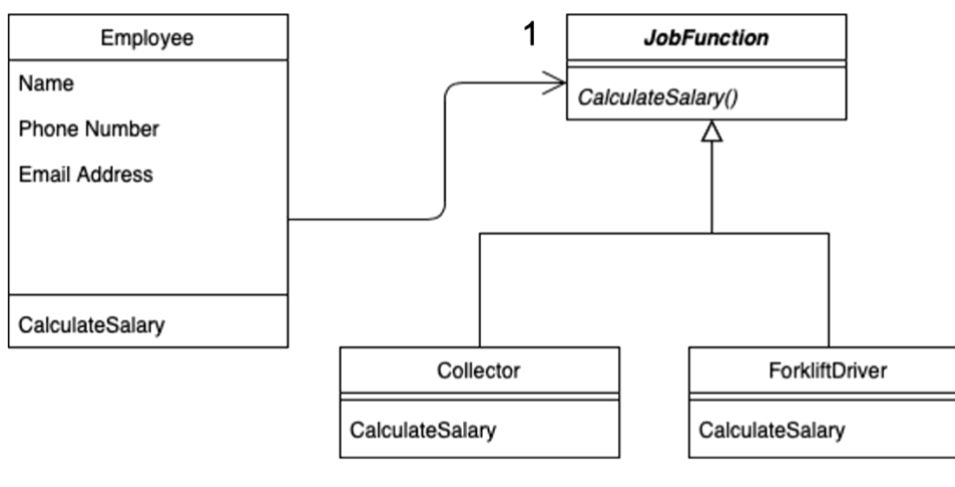


FIGURE 3 CLASS DIAGRAM HOW THE STRATEGY PATTERN POTENTIALLY COULD BE IMPLEMENTED

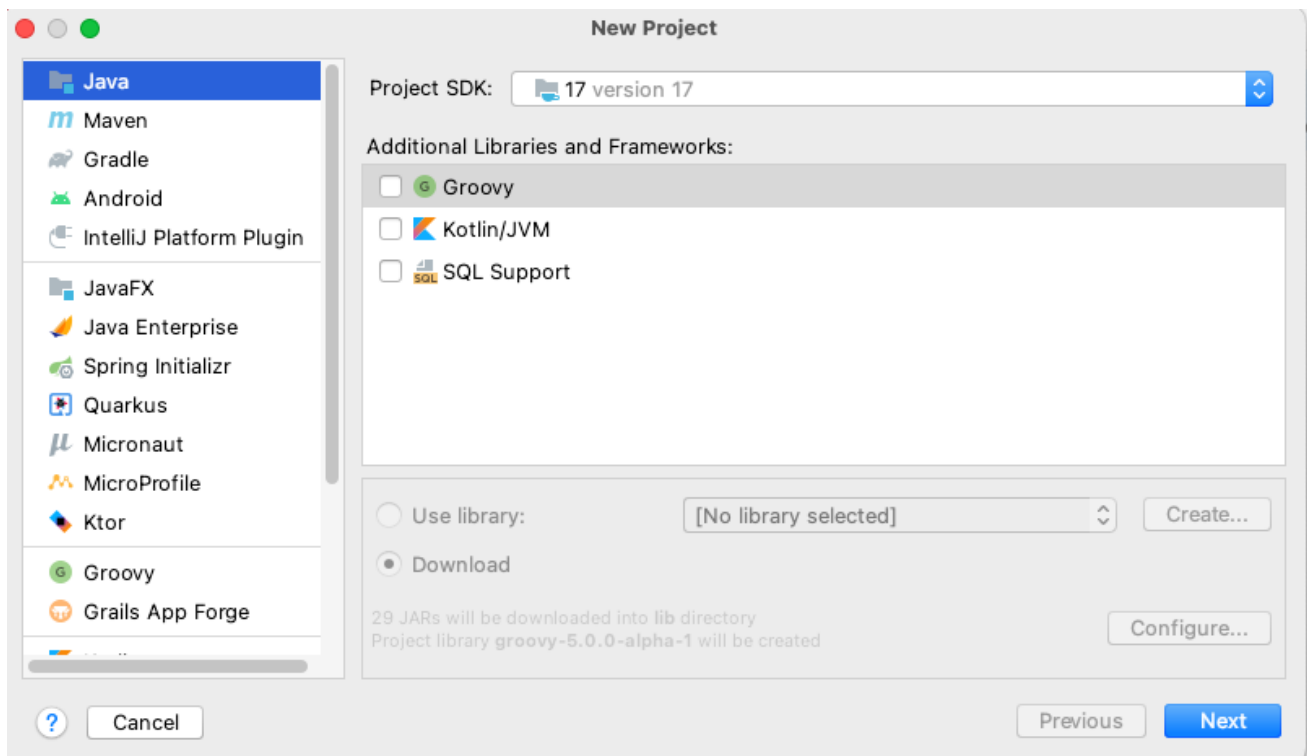


FIGURE 4 – CREATE NEW JAVA PROJECT. NEW PROJECT>SDK 21> NEXT>CHOOSE NAME+LOCATION>FINISH

Assignment 2: Singleton

Use the **Shop** project (available on Brightspace). Analyze the code and write down what the current problem is with the *ShopScanner.addGrocery(..)* and *PrintProductStatistics.printStatistics()* methods. Solve these problems by implementing the Singleton pattern.

Assignment 3: Redesign

Use the footballManager project from last week's workshop assignments and your identified code smells to create a redesign of the code. Create a UML class diagram (or multiple diagrams). Make it clear which decisions you are making to address the underlying problems in the code.

Note: Next week we will actually refactor the project during the workshop, so it's important to complete the redesign this week!

Exam assignment

Create a redesign with one or more class diagrams that resolve the issues identified in your analysis. Consider how you can apply the SOLID principles and, where appropriate, design patterns in your solutions.

Week 7 (Refactoring: iterating)

Theory

Study the material on Refactoring Guru about the Factory pattern.

Also study the SOLID principles that you do not yet fully understand. The SOLID principles are fundamental to most design patterns, so it's important to understand them well in order to apply different patterns effectively.

Note: above all, it's very important to practice a lot with designing, implementing, etc.

Link: <https://refactoring.guru/refactoring>

Workshop

Assignment 1: Factory pattern

Use the 'factory-exercise-PizzaPlace' project, available on Brightspace. Implement the Factory pattern in this project. See further instructions in the README.md file. For this assignment, you will actually (re)write code.

Assignment 2: Refactoring

Use the redesign you created in Assignment 2 from last week (footballManager Java project). Refactor the code based on your redesign. Tip: use the SOLID principles (Single Responsibility, Open/Closed, and Interface Segregation) and, if appropriate, the design patterns you learned.

For this assignment, you will actually modify existing code.

Reminder: All workshop assignments must be signed off in order to be admitted to the assessment (and thus be eligible to receive a grade).

Exam assignment

No new material will follow after this week. With the knowledge you have gained, you should be able to complete exam assignment 2 (and exam assignment 1 if you have not done so yet).

Exam assignment 1 – Battle Engine

This is the first exam assignment. You will work on it during weeks 1–4. The deliverable is a technical design document. No code needs to be submitted for this assignment.

Context

You are working at a game studio that specializes in developing turn-based games. Your team is responsible for the battle engine of a new game. As inspiration, you can look at the battle engine in Dragon Quest XI (<https://youtu.be/-0TW0QxCld8?si=nKFKoXSqJtgn808I&t=30>).

However, the game is always played online. The game company's servers can host multiple instances (server sessions) of the game. A player always plays on a specific instance. If a player wants to play alone, a new (private) instance is started, and the player plays on that instance. However, players can also visit each other's worlds. In that case, a player switches from their current instance to the instance of another player.

When an instance no longer contains any players, that instance is shut down. An instance can contain a maximum of 8 players at the same time.

Because players constantly switch between different instances, no permanent data can be stored on an instance. Therefore, there is also a separate component responsible for storing persistent data, such as the strength of different characters (level), collected weapons, the position in the world, etc.

To play the game, the player must install an application (client) on their own computer.

The game also consists of many other components (such as 3D models, audio, quests, world map, navigation, and equipment). However, you will focus mostly on the **Battle Engine** within the given context.

Assignment

Create a **Technical Design Document** for this system. In this document, you will design the architecture for the above-mentioned (game) system. Use the **C4 model** to show the designs at the appropriate level of detail for each layer. At the deeper layers, the focus should be on the **Battle Engine**. The main responsibility of the Battle Engine is to handle battles between **1–4 player-controlled characters** and **1–4 enemy characters**.

Requirements and Constraints:

- In the Technical Design, work from a global level towards a detailed level using the **C4 model**. At the higher levels, include the entire (game) system (including, for example, the online components). At level 4, only the **Battle Engine** needs to be covered.
- Ensure that the different layers are consistent with each other.
- Part of your design is the **data model**. This data model must include the requirements listed below. Some of these requirements would normally be implemented in code rather than in the data model. Describe in your Technical Design how these requirements could be implemented within your design. You do **not** need to write any code for this assignment. *Example:* In the data model, you need to determine how units, turns, and actions interact with each other. However, the exact implementation of how damage is calculated would probably not be part of the data model. In the document, you could describe that the dealDamage(..) function in your Unit class calculates the damage, etc. However, some design choices may still be required to make the calculation possible.
- For this assignment, you may assume that battles always take place between the player (with a number of characters they control) and a computer-controlled opponent (which also controls a number of characters). Battles do **not** take place between players, and players do not control units together.
- You may use your own interpretations for unclear or incomplete requirements, as long as the choices made are clearly motivated and justified, and do not contradict or override the given requirements. There are often multiple possible ways to design a component; it is important that you can explain how you arrived at your solution.

Assignment: Write a **Technical Design Document** including the following sections:

- Goal: Write in your own words the purpose of the **battle engine**.
- Create a design of the entire system, including the requirements listed below. For each layer create a diagram using the C4 notation, also include the following UML diagrams:
- C4 Layer 1: Context
 - Create **UML use case diagrams** of the actors that use the system.
 - **Tips:**
 - *Provide the reader with the context in which this Battle Engine is used.*
- C4 Layer 2: Container
 - Design the containers that make up the system. Use a **high-level UML deployment diagram and component diagram**. Determine which information is relevant at this level.
- C4 Layer 3: Component
 - Create another **UML deployment diagram and component diagram**, but now at a more detailed level. Determine which information is relevant at this level.
- C4 Layer 4: Code
 - Design the data model using **UML class diagrams**. Design at the appropriate level of detail. You will likely need multiple class diagrams. Clearly indicate the design choices you make using comments/notes.
- **Design Justification:** In this chapter, describe your iterative design process. Consider, for example: your considerations, justification of design choices, alternative approaches considered, adjustments made, insights gained, etc. **Note:** This chapter is important for the assessment of your work.
- General
 - Ensure that the Technical Design contains enough information to reasonably implement the **Battle Engine**.

- You are free to introduce additional containers/components where needed, as long as these choices are clearly described and logically justified.

Requirements

Interface

Because the **battle engine** is part of the larger whole (the game), it must provide the following interface. This allows other subsystems outside the battle engine to communicate with the battle engine via this interface.

I <i>IBattleEngine</i>
● void StartBattle(ConfigBattle) ● Battle GetBattleState() ● List<Action> GetAvailableActions(unitId) ● void ApplyAction(int unitId, Action) ● void NextTurn() ● BattleLog GetBattleLog(battleId)

- void StartBattle(ConfigBattle) - Starts a new battle with 1-4 player characters and 1-4 enemies.
- Battle GetBattleState() - Returns the current status of the battle.
- List<Action> GetAvailableActions(unitId) - Returns the actions a unit is allowed to perform.
- void ApplyAction(int unitId, Action) - Executes an action
- void NextTurn() - Passes the turn to the next player.
- BattleLog GetBattleLog(battleId) - Returns the battle log.

You are free to define the return types and parameters (for example: ConfigBattle, Battle, etc.).

Functional requirements

Turn order

- The battle engine must always have exactly one player selected as the active player (it is this 'playerturn').
- There are 2 players (Human and Computer).
- You may assume that another system outside the battle engine will call `ApplyAction(unitId, Action)` to control the units in a battle. This is out of scope for this assignment. It is likely a system that reads player input (human) or an AI system.
- When `NextTurn()` is called on the battle engine, the other player becomes the active player, and vice versa.
- Only units belonging to the active player can perform actions.

Performing actions

- When `ApplyAction(int unitId, Action)` is called, the `unitId` must match one of the units belonging to the current active player. That unit then performs the provided `Action`.
- An action always occurs between the unit performing it (the caster) and 1..* other units (the targets). This data is provided in the `Action` object.
- The following action types must be implemented:
 - The target(s) take X damage (see "Units" below).
 - The target(s) are healed for X.
 - A stat is increased or decreased by X% (round up).
- An action can also increase or decrease its target(s)' stats by a percentage. (*Example: An action increases the Power stat by 25% on 2 targets: A and B. In this case, the caster's stats do not matter. For example: Target A has Power 8, which becomes $8 * 1.25 = 10$. Target B has Power 1, which becomes $1 * 1.25 = 2$ (1.25 but becomes 2 because we round up)*)

Units

- Each unit has a number of hit points (HP). HP decrease/increase when the unit respectively takes damage or receives healing.
- When HP becomes 0 (or less), the unit is knocked out and must be removed. This unit can no longer perform actions or be a target of other actions.
- Each unit has a maximum number of hit points (max HP). HP can never exceed max HP. (*Example: a unit has 40 HP and 50 max HP, it is healed for 20; although $40 + 20 = 60$, HP cannot exceed max HP, so HP becomes 50.*)
- For calculating damage to units, the system must use 3 stats: Power, Defence, and Spirit. When a unit performs an action that deals damage or healing, these stats are used as follows:
 - Damage
 - Power increases the damage dealt by actions this unit performs.
 - Defence reduces the damage a unit receives when it takes damage.
 - Damage calculation for an action: base damage of the action + caster's Power - target's Defence.
(*Example: An action deals 10 damage to 2 targets: unit A and B. The caster has Power*

4. Target A has Defence 6 and receives $10+4-6 = 8$ damage; Target B has Defence 1 and receives $10+4-1 = 13$ damage.)

- Healing
 - Spirit increases the amount of healing done by actions this unit performs.
 - Healing calculation for an action: base healing of the action + caster's Spirit.
(Example: An action heals 3 targets A, B, and C for 15. The caster has Spirit 6. A, B, and C are each healed for $15+6 = 21$. Their other stats do not matter.)

Starting a battle or requesting battle state

The system must be able to either provide the state to start a battle, or return the current state of a battle: `StartBattle(..)` and `Battle GetBattleState()`. It is important that the current status of a battle is provided or returned completely.

The function `GetBattleLog(..)` must return a log of the conducted battle, which could look as follows:

- Battle starts
- Turn passed to player 1 (Human)
- Unit A uses Action X1 and did 15 damage to unit B.
- Unit A uses Action X1 and did 12 damage to unit C
- Unit D uses Action X2 and heals unit A for 12.
- Turn passed to player 2 (AI)
- Unit B uses Action X2 and did 3 damage to unit A and 4 damage to unit D.
- Turn passed to player 1
- ..

Exam assignment 2 – 3Dprintscheduler



In this exam assignment, existing code will be analyzed, redesigned, and eventually refactored. The code used is the ‘3DPrintScheduler’ software, which can be found on Brightspace. Additionally, the ‘3D Printer case study’ from the next chapter in the workbook will be used.

This exam assignment consists of 2 parts:

1. Create a **technical design** focused on analysing and redesigning the existing ‘3DPrintScheduler’ code.
2. After completing the redesign, refactor (modify) the ‘3DPrintScheduler’ software, this means you will actually **write code**.

You will deliver the following **2 deliverables** for this exam assignment:

1 – Technical Design, which includes:

- This assignment focuses primarily on **Layer 4 of the C4 model**. However, as software systems become more complex, it is important to understand where a component is located within the overall system architecture. Therefore, also briefly describe the first three layers of the system (using **C4 notation**) as they apply to this exam assignment. You may introduce additional components if needed, as long as they are logical and properly justified.
- **Analyze the existing code (3dprintscheduler)** and make the current structure of the code clear.

- **Tips:** Use, for example, multiple class diagrams with notes. Note: the goal is not simply to create a class diagram (as this can be generated automatically), but to gain insight into (and demonstrate) how the code is structured and how the different parts relate to each other.
- Describe the identified **code smells**.
- Describe the **structural problems** in the existing code.
- Create a **redesign** of the code so that it becomes more extensible and maintainable. The **SOLID principles** and supporting **design patterns** are leading principles for this redesign. Redesign **Layer 4** (and optionally Layer 3). For layer 4 Use UML class diagrams that fit the different levels of detail and/or components. Ensure that the design is complete enough to use as a basis for implementing the refactor.
- **(Added after the refactor):** Validate that the code works correctly (through testing).
- **Design Justification:** Although a good design and code quality are important, it is equally important to demonstrate the design process. Therefore, add comments/notes/annotations (or, for example, a log) that provide insight into your design decisions. Describe, for example, the considerations made, justification of design choices, alternative approaches considered, adjustments made, and insights gained during the design process. **Note:** This chapter is very important for the assessment of your work.

2 – Refactored code

Requirements after refactoring:

- a. The major issues have been identified and addressed.
- b. The code has been improved, making it at least more extensible and maintainable.
- c. The SOLID principles (and, where applicable, design patterns) have been correctly applied.
- d. The code still behaves in the same way as it did before the refactoring.

3D printer casus

(use this case for exam assignment 2)

Introduction

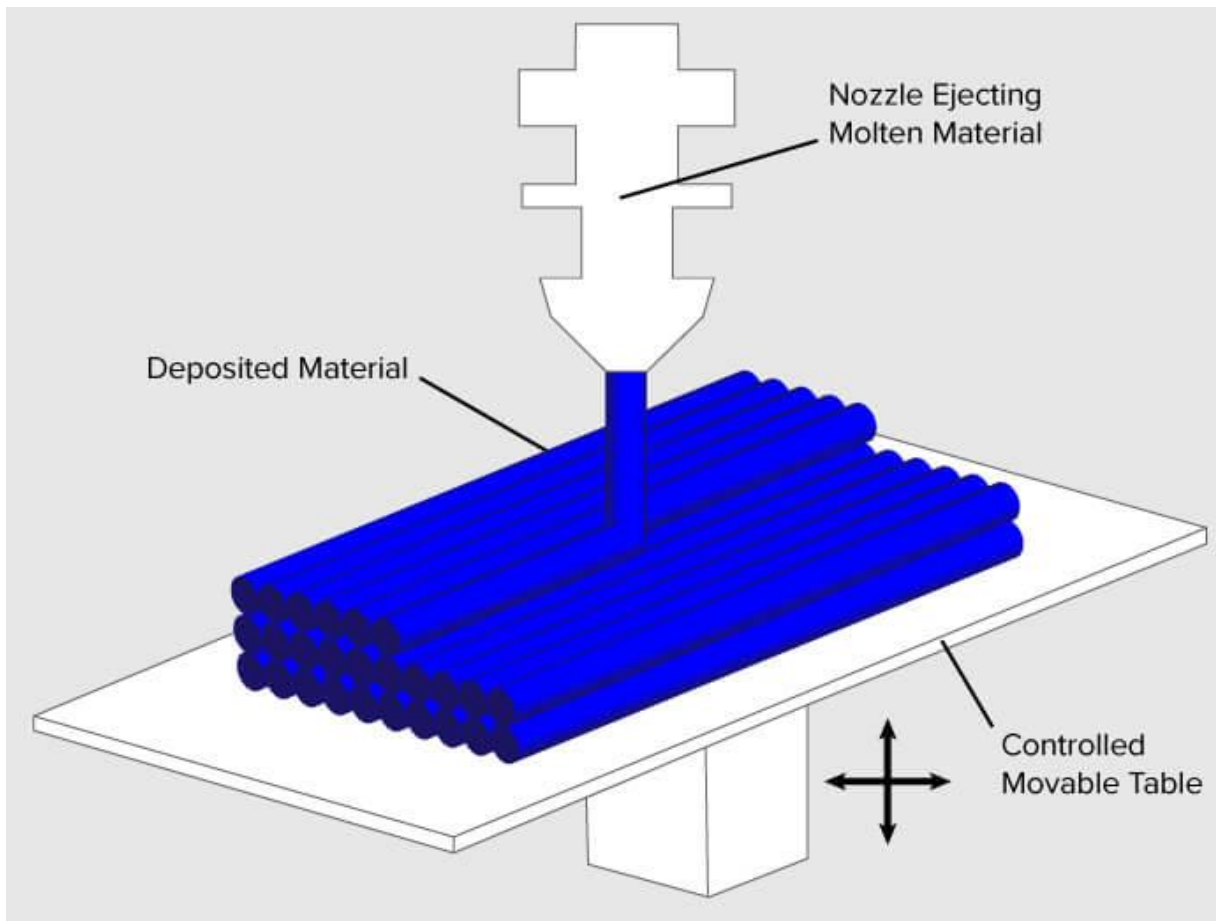
Jack started with a hobby of 3D printing with a single printer. Quickly friends and family started requesting various prints for which they paid, resulting in small profit. This allowed him to buy another printer, and another, and soon he was operating a small print farm. Offering his services online and via folders allowing him to grow the business. As the business grew, he also got different printers and modified some. Allowing him to print multi-color and print in special materials.

Quickly it got to a point where he felt printers were not running at the most optimal rate. This was mainly because he just printed them in order of arrival which sometimes meant swapping the spools multiple times in a day. While completing prints and handling printers still required manual activity, he figured he could optimize printing activity by writing a piece of software that allowed him to properly schedule the printers.

In creating this program and assignment certain liberties were taken to have an assignment that is both close to reality and still workable as a school assignment. Please keep this in mind when examining the working of this program.

Background information

FDM printers operate using large spools of filament which are fed through a tube to a hot end. The hot end will melt the filament and then using X, Y and Z coordinate instructions move along a set path. This results in an object being printed layer by layer.



A spool of a kilogram tends to have about 330 meters. A small print takes up about 2–3 meters of filament. Though this can vary from print to print.

PLA and PETG are the most common filaments. Where PETG requires the printer to run at higher temperatures. ABS requires a printer that is housed to prevent warping, ABS is resistant to higher temperatures. Though a printer can easily be placed in housing without much trouble. Similarly, printers can also be upgraded to carry more colors.

Replacing a spool can easily take a few minutes as the printer has to heat up separately, the old spool must be removed and then a new spool must be prepared and added. The less often this needs to be done, the better.

Basic program operation

The printers are not connected to any other computer. They each have a micro-SD card containing all the prints. Jack must manually start each printer and clear the print from the printer.

The program is designed to tell Jack what to print next when a print is done. This saves him time to find out what would be more optimal. For instance, if there are multiple prints using blue PLA in the queue, then the program will favor those so that he doesn't have to change spools constantly.

When the printer tells Jack what to print it assumes that print will happen. When a print is done, he tells the program a print is done. On completion the computer immediately tells him which print to start next on that printer. It will also tell him if he needs to swap out a filament spool for the next print.

Currently, if a print task fails, Jack still marks the printer as done and it will follow the usual process. He will then manually add the print to the queue again. This is not an ideal situation and something that needs to be fixed in the future.

For convenience all possible prints and printers are managed in JSON files. The reason for this is that adding printers to the system is not that exciting and doesn't happen a lot. The JSON contains the name of the print, GCODE filename, and printing time. This estimated printing time is generated by the slicing software (software that generates the Gcode file).

When adding a printing task to the system the print is chosen and the filament is chosen for that print. This will add it to the queue. The system should choose prints that are viable to do and ideally don't require a filament change, unless there is no other choice.

Controlling the program is done almost exclusively with the number pad for convenience. This is an important requirement for Jack to make using the program easier.

In the future Jack would like to introduce an alternative method of scheduling where small remaining spools can be used up efficiently.

Requirements

To assist himself in writing the software he created the following requirements for himself. Since he is the only one using this system, he has kept it simple and not too complicated. Favoring a program that can run on a single computer and runs a simple text interface so it can be run on a small computer in the shed dedicated to the print farm. Additionally, all requirements are must as the focus was on what was specifically needed.

Business requirements

Code	Description
B1	Spend less time figuring which print to start next.
B2	Maintain an overview of all active printers.
B3	Maintain an overview of all print tasks.
B4	Get an overview of which spools need to be ordered.
B5	Have a central list of all available prints.
B6	Be able to get the most out of leftover printing spools

User requirements

Code	Description	Source
U1	Add new printer to the system	B2
U2	Add new print to the system	B2
U3	Add a print task to the system	B3
U4	Register a print completion	B1
U5	Be informed on which printer to start a print task.	B1
U6	Operation should be done using menu numbers for quick operation.	B1
U7	Get information all printers and current spools and prints	B3
U8	Get a list of all spools and their current length.	B4
U9	Have a central list of all available prints.	B5
U10	Be able to select a printing strategy related to spool usage.	B6

System Requirements

Code	Description	Source
NF1	Runs on one computer.	
NF2	Does not need to interact with the printers directly.	
NF3	Data is stored locally in simple files.	
NF4	Interface is entirely text based.	

Code	Description	Source
F1	Display a menu of options with number input.	U6
F2	List all printers with their current spools and jobs.	U7
F3	List all available prints	U9
F4	List all available Spools and their current length	U8
F5	Add task to queue using number pad. Selecting print, filament type and color.	U3, U6
F6	Start print queue which will try to assign a print to each printer.	U5
F7	Register print completion which reduces spool length and selects a new print.	U4, U5

F8	Register print failure which returns the print to the queue, reduces spool length, and selects a new print.	U5
F9*	Select a different printing strategy.	U10

Testing

Jack never wrote any tests for his program, mainly because he couldn't figure out how to, given the current code structure. There is a Markdown file in the project explaining what tests should exist and succeed. There is also a table showing which print fits on which printer. This can be used to ensure tests are successful.

Diagrams

In figure 1 we see a general overview of how the program is used. Once the program is started and things are added to the queue, it waits for Jack to register a completion of a print.

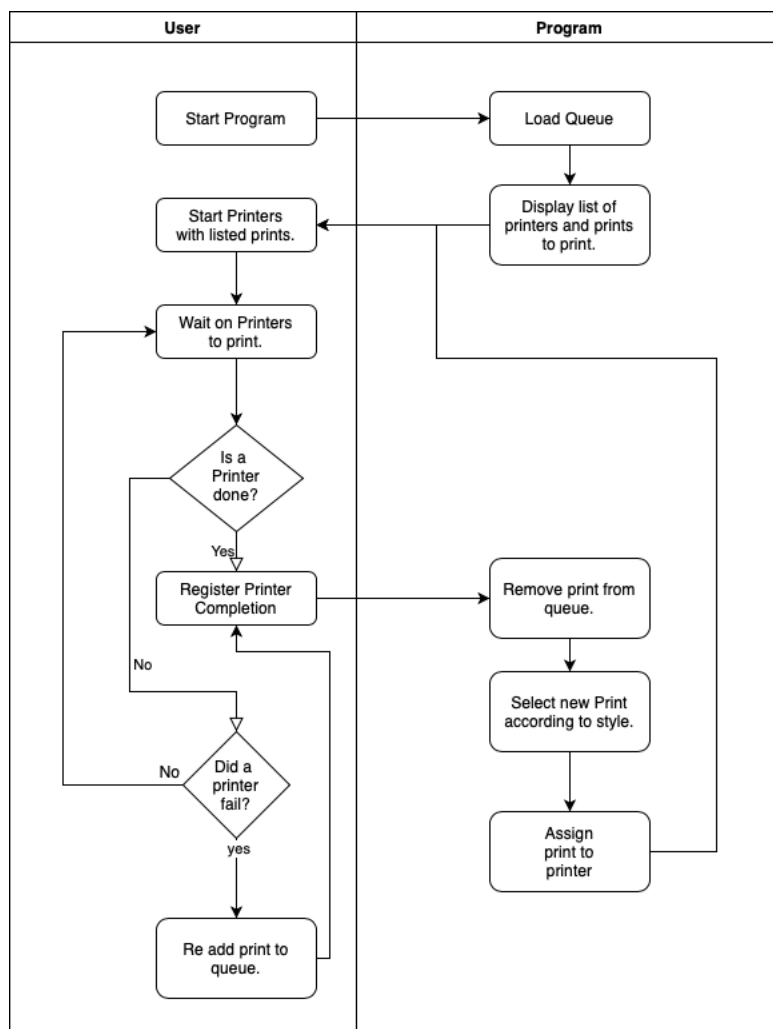


FIGURE 2 GENERAL OPERATING PROCEDURE

Reflection Report

Software design is an iterative process. Therefore, it is important to justify the choices you make and to regularly reflect on your own actions and decisions. In the reflection report, you should address the topics below and demonstrate how you have incorporated your insights into the next phase of the design process.

Submit an **individual** partial reflection every two weeks through the eJournal environment. The purpose is to enable you to immediately apply insights gained during the design process.

Time-Based Evidence: Although it is not recommended, you may deviate from the suggested submission dates. In that case, the responsibility for providing time-based evidence lies with you. In your partial reflections, you must clearly explain:

- When the reflection moments took place (including dates).
- Which insights resulted from those reflections and at what moments.
- When and how you applied those insights.

If you submit the partial reflections before the recommended deadlines, this time-based evidence requirement no longer applies.

Regardless of when you submit them, you must always submit **three partial reflections**.

RECOMMENDED SUBMISSION DATES

- **Partial Reflection 1:** Covers weeks 1–2, submit before the end of week 3.
- **Partial Reflection 2:** Covers weeks 3–4, submit before the end of week 5.
- **Partial Reflection 3:** Covers weeks 5–6, submit before the end of week 7.

End of week X = before the last Sunday (23:59) of that week.

Note: After submitting all three partial reflections in the eJournal environment, you must submit your portfolio for assessment. See the chapter “eJournal Environment → Step 5” below for details.

Partial Reflection

In each partial reflection, answer the following questions:

Design and Implementation: Were there any changes to the design or code? Which changes did you make, why were they necessary, and what did you learn from them?

Collaboration: How did you experience the collaboration? What went well, what went less well, and what would you approach differently in the next period?

Research and Professional Development: **Which knowledge or skills were you lacking during this period? How did you deal with this, and how do you plan to further develop these skills?**

Progress and Insights: **Which insights did you gain during this period? Which insights from your previous reflection did you actually apply, what were the results, and which concrete points of attention will you take with you into the next period?**

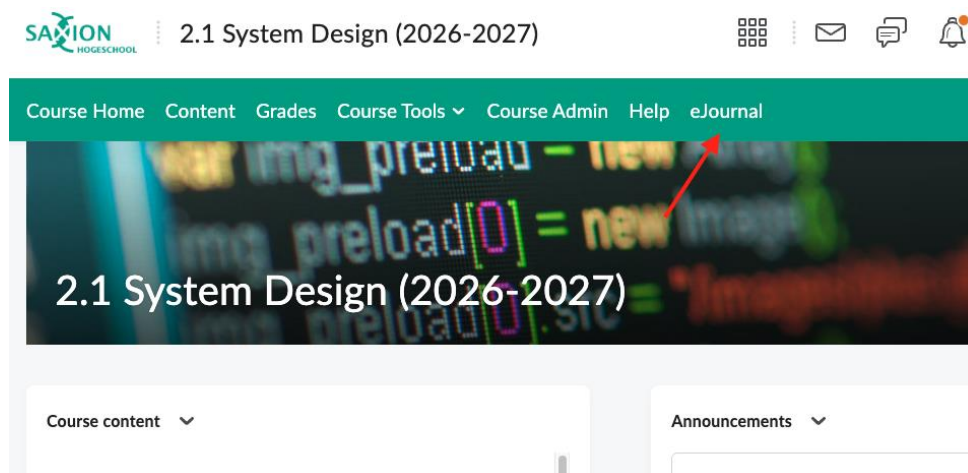
eJournal Environment

You submit your reflections through the **eJournal environment**. This is the online platform where, throughout your education, you collect all work related to professional skills across modules and academic years. In this way, you build a personal portfolio and gain increasing insight into your strengths and areas for development.

Because this environment is still relatively new, an explanation of how it works is provided below. If you encounter problems or find anything unclear, please contact your **Systems Design teacher** as soon as possible.

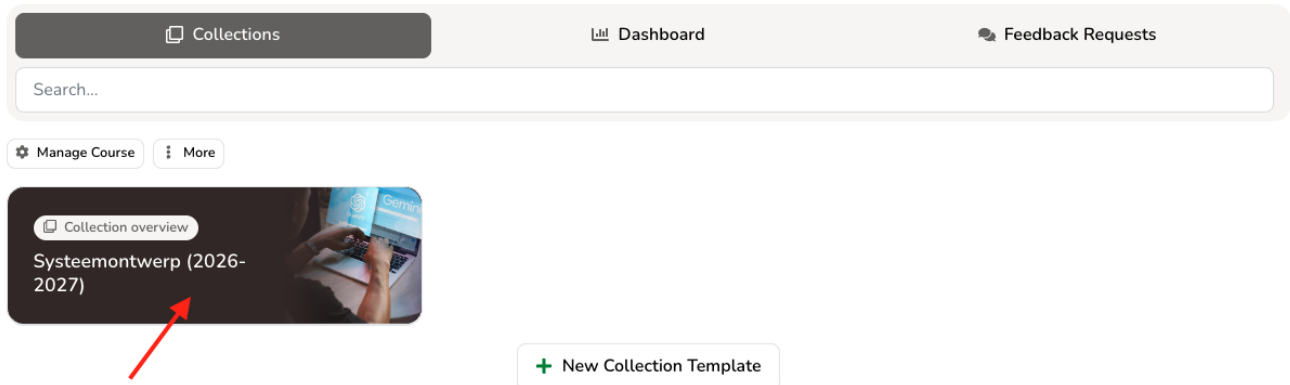
How to submit your work in the eJournal environment

Step 1: In the Brightspace environment of System Design, click 'eJournal' in the main menu.

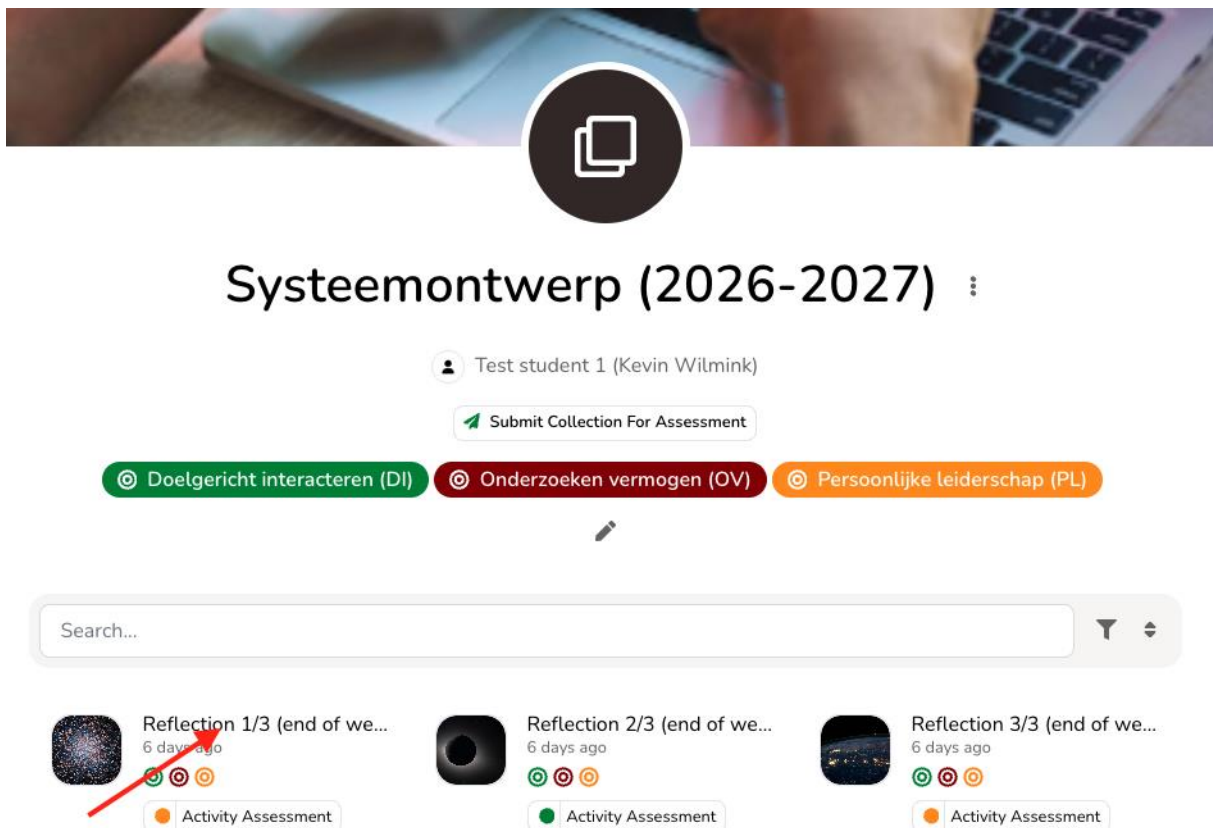


Step 2: Click on the collection *System design*

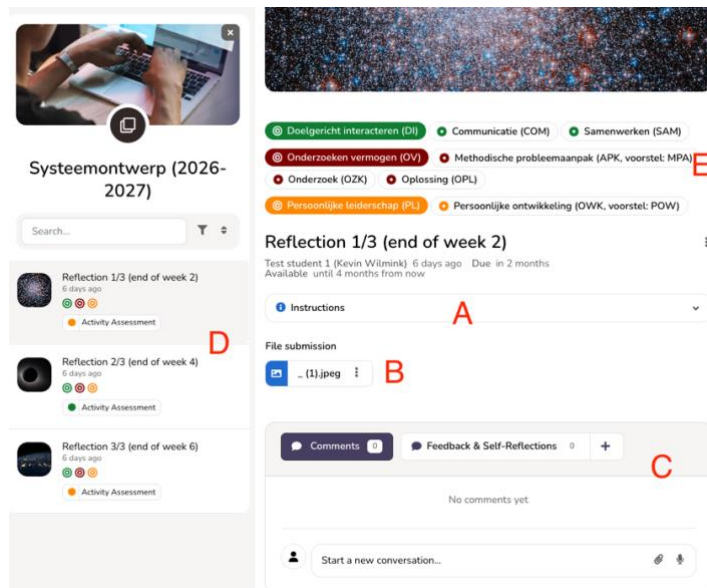
2.1 Steemontwerp (2026-2027)



Step 3: To submit partial-reflection 1, click submission point 1



Step 4: See below for more information on how to submit your work:



A: The instructions for this assignment.

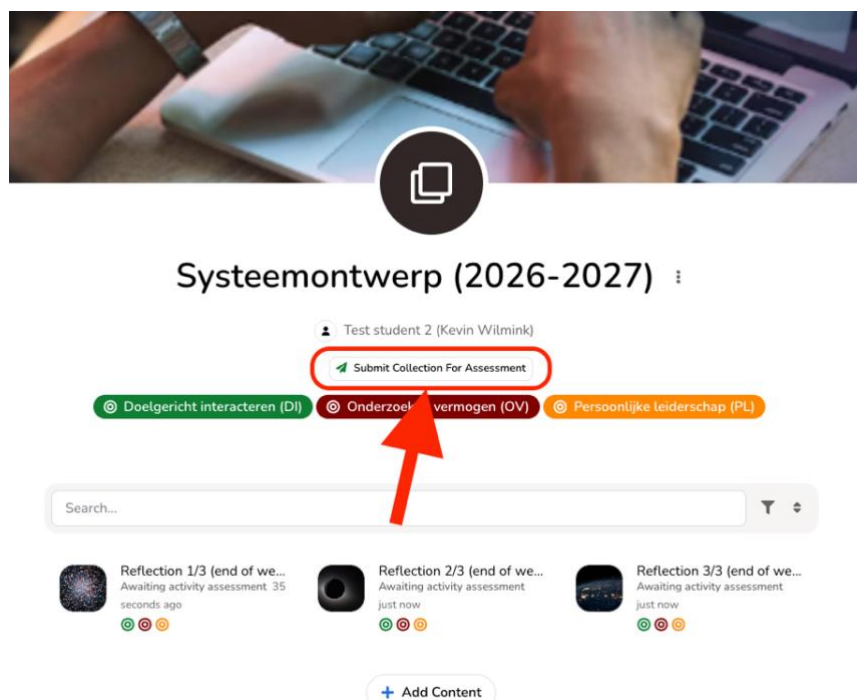
B: The file you uploaded. Alternatively, you can select a file to upload here (upload your report as a PDF file).

C: Option to provide feedback or add comments (optional).

D: Quick navigation between the other submission points.

E: Overview of the professional skills to which this assignment is linked.

Step 5: Once you have submitted all three partial reflections, the **portfolio must be submitted as a whole** before it can be assessed. You can do this by clicking “*Submit Collection for Assessment*” on the main screen.



Bibliography

HBO-i, s. (2025, September). *Domeinbeschrijving*. Opgehaald van HBO-i: <https://www.hbo-i.nl/publicaties-domeinbeschrijving/>